

Introdução e Conceitos

Antes de começar, é importante entender alguns conceitos básicos do ASP.NET Core e do Entity Framework Core:

- **DbContext:** representa a sessão de trabalho com o banco de dados; permite consultar e salvar dados.
- **DbSet:** representa uma tabela no banco; cada DbSet no DbContext corresponde a uma entidade.
- **Model:** classe que representa os dados de uma tabela (propriedades mapeiam colunas).
- **Controller:** recebe requisições HTTP e retorna respostas; normalmente faz CRUD usando o DbContext.
- **appsettings.json:** arquivo de configuração do projeto; guarda connection strings, níveis de log e outras definições.
- **Migration:** histórico de alterações no banco de dados; permite criar e atualizar tabelas sem perder dados.

Checklist Genérico para Criar uma API com Banco de Dados

- Instalar as ferramentas necessárias para conectar ao banco de dados.
- Criar as classes de entidade que representam as tabelas.
- Configurar o DbContext e registrar no container de dependências.
- Adicionar a connection string do banco no arquivo de configuração.
- Criar migrations para refletir as tabelas no banco.
- Criar controllers para expor endpoints HTTP de consulta e alteração dos dados.
- Testar a API localmente usando Swagger ou Postman.
- Publicar a API e ajustar a connection string para o ambiente de produção.

Acesso ao Console / Terminal

Algumas etapas do guia exigem executar comandos, seja para criar migrations ou atualizar o banco de dados. Você pode fazer isso de duas formas:

- **Terminal do Windows / CMD / PowerShell:** Abra o menu iniciar, digite `cmd` ou `PowerShell` e pressione Enter. Navegue até a pasta do projeto usando `cd` e execute os comandos.
- **Package Manager Console (PMC) do Visual Studio:** No Visual Studio, vá em **Tools > NuGet Package Manager > Package Manager Console**. Isso abre um console dentro do Visual Studio, já com o diretório correto do projeto.

Para comandos do .NET CLI, você pode usar o terminal externo; para comandos do Entity Framework (como `Add-Migration` e `Update-Database`) o PMC é mais conveniente.

1. Instalar Pacotes via Visual Studio

No Solution Explorer, clique com o botão direito sobre o projeto e selecione **Manage NuGet Packages**. Na aba **Browse**, procure e instale:

- **Microsoft.EntityFrameworkCore.SqlServer**
- **Microsoft.EntityFrameworkCore.Tools**

2. Criar o Model (PetModel)

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace BichosApi.Models
{
    public class PetModel
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int Id { get; set; }

        [Required]
        public string Nome { get; set; } = string.Empty;

        public string Especie { get; set; } = string.Empty;

        public DateTime DataNascimento { get; set; }
    }
}
```

3. Criar o DbContext

```
using Microsoft.EntityFrameworkCore;
using BichosApi.Models;

namespace BichosApi.Data
{
    public class BichosDbContext : DbContext
    {
        public BichosDbContext(DbContextOptions<BichosDbContext> options)
            : base(options) { }

        public DbSet<PetModel> Pet { get; set; }
    }
}
```

4. Configurar appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\
\\mssqllocaldb;Database=DBBichos;Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

5. Configurar Program.cs

```
using BichosApi.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddDbContext<BichosDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultCo
nnection")));

var app = builder.Build();

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

6. Criar Migration e Atualizar Banco

```
Add-Migration PrimeiraMigration
Update-Database
```

Nota: Execute esses comandos no Package Manager Console do Visual Studio ou no terminal do Windows.

7. Criar Controller

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using BichosApi.Data;
using BichosApi.Models;

namespace BichosApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class PetController : ControllerBase
    {
        private readonly BichosDbContext _context;

        public PetController(BichosDbContext context)
        {
            _context = context;
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<PetModel>>> GetPets()
        {
            return await _context.Pet.ToListAsync();
        }

        [HttpPost]
        public async Task<ActionResult<PetModel>> PostPet(PetModel petModel)
        {
            _context.Pet.Add(petModel);
            await _context.SaveChangesAsync();
            return CreatedAtAction(nameof(GetPets), new { id = petModel.Id }, petModel);
        }
    }
}
```

8. Teste Local

```
POST /api/Pet
{
  "Nome": "Fido",
  "Especie": "Cachorro",
  "DataNascimento": "2020-01-01"
}
```

Conclusão

- Banco local `DBBichos` criado com EF Core
- API mínima funcional
- Possibilidade de criar/ler registros da tabela `Pet`
- Base pronta para subir no MonsterASP depois de ajustar connection string e publicar a pasta `publish`